



Environment Variables & Secrets

Cypress tests often need values that change across environments like API URLs, credentials, feature flags, or configuration toggles.

Not all values should be treated the same. Some are **secrets that must be protected**, while others are **safe to expose** and convenient to access synchronously.

This guide helps you choose the right API for your use case.

Secrets and sensitive values

Use `cy.env()` for sensitive values like API keys, passwords, tokens, or credentials.

`cy.env()` retrieves only the values you explicitly request, when you need them, and avoids exposing all environment variables in browser state. This provides privileged access that doesn't automatically serialize values into browser context.

Examples of secrets:

- API keys
- Authentication tokens
- Passwords
- Database credentials
- Private service endpoints

```
cy.env(['apiKey']).then(({ apiKey }) => {  
  cy.request({  
    url: 'https://api.example.com/data',  
    headers: { Authorization: `Bearer ${apiKey}` },  
  })  
})
```



```
  })  
})
```

See the `cy.env()` command documentation for complete details.

Public configuration values

Use `Cypress.expose()` for public, non-sensitive configuration values.

`Cypress.expose()` provides synchronous access to configuration that is safe to expose in the browser context. Values are accessible to application code, third-party scripts, and browser extensions.

Examples of public configuration:

- Feature flags
- API versions
- Plugin configuration
- Environment labels (staging, prod)
- Public service URLs

```
const apiVersion = Cypress.expose('apiVersion') // Synchronous  
access  
const featureFlag = Cypress.expose('featureFlag')  
  
if (featureFlag) {  
  cy.get(`[data-  
testid="feature-${apiVersion}"]`).should('be.visible')  
}
```

See the `Cypress.expose()` API documentation for complete details.

Set environment variables

Environment variables for `cy.env()` can be set using several methods:



1. **Cypress configuration file** - Set in the `env` key of your Cypress configuration
2. **`cypress.env.json` file** - Create a `cypress.env.json` file in your project root
3. **`CYPRESS_*` environment variables** - Set OS-level environment variables with `CYPRESS_` prefix
4. **`--env` CLI flag** - Pass environment variables via command line
5. **`setupNodeEvents`** - Set dynamically in the `setupNodeEvents` function

1. Configuration File

Set environment variables in your Cypress configuration file under the `env` key:

`cypress.config.js` `cypress.config.ts`

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  env: {
    apiUrl: 'https://api.example.com',
    apiKey: process.env.API_KEY, // From OS environment
  },
})
```

2. `cypress.env.json`

Create a `cypress.env.json` file in your project root. Values here override conflicting environment variables in your Cypress configuration.

```
{
  "host": "veronica.dev.local",
  "api_server": "http://localhost:8888/api/v1/"
}
```

Important: Add `cypress.env.json` to `.gitignore` if it contains sensitive



data.

3. `CYPRESS_*` environment variables

Set OS-level environment variables with the `CYPRESS_` or `cypress_` prefix:

```
export CYPRESS_HOST=laura.dev.local
export cypress_api_server=http://localhost:8888/api/v1/
```

Cypress automatically removes the leading `CYPRESS_` or `cypress_` prefix and normalizes the name.

The environment variable `CYPRESS_INTERNAL_ENV` is reserved and should not be set.

4. `--env` CLI flag

Pass environment variables via the command line. Multiple values must be separated by a comma, not a space. In some shells, like Windows PowerShell, you may need to surround the key/value pair with quotes.

```
cypress run --env
host=kevin.dev.local,api_server=http://localhost:8888/api/v1
```

5. `setupNodeEvents`

Set environment variables dynamically in the `setupNodeEvents` function:

`cypress.config.js` `cypress.config.ts`

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  // setupNodeEvents can be defined in either
  // the e2e or component configuration
```



```
e2e: {
  setupNodeEvents(on, config) {
    export default defineConfig({
      e2e: {
        setupNodeEvents(on, config) {
          config.env.apiKey = process.env.API_KEY
          return config
        },
      },
    })
  },
},
})
```

Set exposed configuration

Exposed configuration for `Cypress.expose()` can be set via:

1. **Cypress configuration file** - Set in the `expose` key of your Cypress configuration
2. `--expose` **CLI flag** - Pass exposed configuration via command line

1. Configuration file

Set exposed configuration in your Cypress configuration file under the `expose` key:

cypress.config.js

cypress.config.ts

```
const { defineConfig } = require('cypress')

module.exports = defineConfig({
  expose: {
    apiVersion: 'v2',
    featureFlag: true,
    environment: 'staging',
  },
})
```



```
},  
})
```

2. `--expose` CLI flag

Pass exposed configuration via the command line:

```
cypress run --expose apiVersion=v2,featureFlag=true
```

See the [Cypress.expose\(\)](#) API documentation for complete details on setting exposed configuration.

Migrate from Cypress.env()

If you're using the deprecated `Cypress.env()` API, migrate to the appropriate modern API:

- **Sensitive values** → [cy.env\(\)](#)
- **Public configuration** → [Cypress.expose\(\)](#)

See the [Migration Guide](#) for detailed migration instructions.

See also

- [cy.env\(\)](#)
- [Cypress.expose\(\)](#)
- [Configuration](#)
- [Best Practices: Handling Secrets](#)

Last updated on **Feb 3, 2026**

CONTENTS

